

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO GIỮA KỲ

Môn: Ứng dụng trí tuệ nhân tạo cho ngôn ngữ

**END-TO-END-NLP
SYSTEM BUILDING**

Nhóm sinh viên thực hiện

Nguyễn Công Cường	23020376
Nguyễn Đức Huy	23020376
Nguyễn Trần Huy	23020376
Ngô Đình Linh	23020376

Giảng viên hướng dẫn: ThS. Ngô Minh Hương

Hà Nội, tháng 5 năm 2026

Mục lục

1	Giới thiệu	2
1.1	Bối cảnh và yêu cầu bài tập	2
1.2	Phạm vi nội dung	2
1.3	Hướng tiếp cận	2
1.4	Mục tiêu thực hiện	3
2	Xây dựng dữ liệu	4
2.1	Thu thập và tiền xử lý dữ liệu	4
2.1.1	Nguồn dữ liệu	4
2.1.2	Tiền xử lý tài liệu	4
2.2	Tạo tập câu hỏi – câu trả lời	5
2.2.1	Quy trình tạo câu hỏi	5
2.2.2	Phạm vi câu hỏi	5
2.3	Kiểm tra chất lượng dữ liệu	5
3	Phân công việc đã thực hiện	7
3.1	Sử dụng Embedding MiniLM + QA DistilBERT	7
3.2	Cách hoạt động chi tiết	7
3.3	Đánh giá kết quả	8
3.4	Sử dụng BGE Embedding + QA RoBERTa với hybrid reranking	8
3.5	Sử dụng E5 Embedding + QA RoBERTa	9
3.6	TF-IDF RAG Baseline	10
3.7	So sánh kết quả các hệ thống	11
3.8	Nhận xét lỗi thường gặp	12
	Đóng góp của các thành viên	13

1. Giới thiệu

1.1 Bối cảnh và yêu cầu bài tập

Bài tập này yêu cầu xây dựng một hệ thống xử lý ngôn ngữ tự nhiên end-to-end cho bài toán trả lời câu hỏi thực tế (factual question answering). Với mỗi câu hỏi đầu vào, hệ thống cần đưa ra một câu trả lời ngắn gọn, chính xác và có thể đối chiếu với đáp án tham chiếu. Dữ liệu đầu vào được lưu trong file `questions.txt`, mỗi dòng là một câu hỏi; đầu ra của hệ thống được lưu trong file `system_output.txt`, mỗi dòng là câu trả lời tương ứng với câu hỏi cùng vị trí.

Khác với các bài toán hỏi đáp mở thông thường, yêu cầu của bài tập nhấn mạnh việc tự xây dựng dữ liệu và kho tri thức cho miền câu hỏi cụ thể. Vì vậy, nhóm không chỉ cần chạy một mô hình có sẵn, mà còn phải thu thập tài liệu công khai, xử lý dữ liệu thô, tạo bộ câu hỏi – câu trả lời, xây dựng hệ thống truy xuất và đánh giá kết quả. Các độ đo chính gồm Exact Match, token-level F1 và Answer Recall; do đó, câu trả lời của hệ thống cần đúng trọng tâm và hạn chế thêm các thông tin không được hỏi.

1.2 Phạm vi nội dung

Trong phạm vi bài tập, nhóm lựa chọn miền thông tin liên quan đến Đại học Quốc gia Hà Nội (VNU) và một số trường thành viên như Trường Đại học Công nghệ (UET), Trường Đại học Ngoại ngữ (ULIS) và Trường Đại học Khoa học Xã hội và Nhân văn (USSH). Các câu hỏi được xây dựng xoay quanh những thông tin có thể kiểm chứng từ tài liệu nguồn, ví dụ lịch sử hình thành, tuyển sinh, điểm chuẩn, học phí, chương trình đào tạo, quy chế đào tạo, chỉ tiêu tuyển sinh và tổ hợp xét tuyển.

Những thông tin này phù hợp với bài toán RAG vì chúng khá chuyên biệt và thường thay đổi theo từng năm. Một mô hình ngôn ngữ chỉ dựa vào kiến thức đã học trước đó có thể trả lời thiếu chính xác hoặc không cập nhật, đặc biệt với các câu hỏi về học phí, điểm chuẩn hoặc phương thức tuyển sinh. Việc bổ sung tài liệu liên quan trước khi trả lời giúp hệ thống có cơ sở rõ ràng hơn và giảm nguy cơ sinh ra thông tin không có trong nguồn.

1.3 Hướng tiếp cận

Theo yêu cầu của đề bài, nhóm triển khai hệ thống theo hướng Retrieval-Augmented Generation (RAG). Ở mức tổng quát, hệ thống trước tiên biểu diễn câu hỏi và tài liệu dưới dạng có thể so sánh được, sau đó truy xuất các đoạn tài liệu liên quan nhất với câu hỏi. Các đoạn được truy xuất tiếp tục được dùng làm ngữ cảnh cho bước đọc hiểu hoặc sinh câu trả lời.

Cách tiếp cận này phù hợp với factual QA vì chất lượng câu trả lời phụ thuộc nhiều vào khả năng tìm đúng bằng chứng trong tài liệu. Trong bài làm này, kho tài liệu được xây dựng từ các nguồn công khai về VNU/UET và các trường thành viên. Từ kho tài

liệu đó, hệ thống RAG gồm ba thành phần chính: bộ biểu diễn câu hỏi và tài liệu, bộ truy xuất tài liệu, và bộ đọc để tạo câu trả lời cuối cùng.

1.4 Mục tiêu thực hiện

Mục tiêu của bài làm là hoàn thành một quy trình hỏi đáp có thể chạy từ dữ liệu đầu vào đến kết quả đầu ra theo đúng định dạng được yêu cầu trong đề bài. Nhóm thực hiện các bước chính gồm thu thập và làm sạch tài liệu, tạo bộ dữ liệu train/test, kiểm tra chất lượng annotation, xây dựng các biến thể RAG, sinh câu trả lời cho tập test và đánh giá bằng các độ đo được yêu cầu.

Một phần quan trọng của bài làm là bộ dữ liệu do nhóm tự tạo. Các câu hỏi và đáp án được xây dựng từ các nguồn công khai, có thể kiểm chứng, và được rà soát thủ công để đảm bảo câu hỏi khớp với đáp án. Bộ dữ liệu này vừa phục vụ phát triển hệ thống, vừa dùng để phân tích chất lượng của các phương pháp RAG được thử nghiệm trong bài.

2. Xây dựng dữ liệu

2.1 Thu thập và tiền xử lý dữ liệu

2.1.1 Nguồn dữ liệu

Nhóm xây dựng bộ tài liệu từ các nguồn công khai liên quan đến VNU và các trường thành viên như UET, ULIS và USSH. Khi lựa chọn nguồn, nhóm ưu tiên các website chính thức của trường hoặc các trang công khai có thông tin rõ ràng về tuyển sinh, học phí, chương trình đào tạo và quy chế đào tạo. Việc ưu tiên các nguồn đáng tin cậy giúp đảm bảo rằng đáp án trong bộ dữ liệu có thể được kiểm chứng lại khi cần.

Các nguồn dữ liệu được chọn để bao phủ nhiều nhóm thông tin khác nhau. Nhóm thu thập các trang giới thiệu trường để lấy thông tin về lịch sử, tên gọi, địa điểm và thông tin tổ chức. Các trang tuyển sinh được dùng để lấy điểm chuẩn, chỉ tiêu, tổ hợp xét tuyển và phương thức xét tuyển. Ngoài ra, nhóm cũng thu thập thông tin về học phí, chương trình đào tạo, loại bằng cấp, thời gian học, số tín chỉ và quy chế học tập.

Sau khi xử lý, kho dữ liệu gồm 29 tài liệu nguồn và 461 đoạn truy xuất. Các tài liệu này là cơ sở để xây dựng câu hỏi, tạo tập train/test và phục vụ hệ thống RAG ở bước truy xuất.

2.1.2 Tiền xử lý tài liệu

Nhóm sử dụng file Python `build_manual_dataset.py` để hỗ trợ quá trình xử lý dữ liệu. File này có nhiệm vụ trích xuất nội dung văn bản từ tài liệu HTML/PDF, loại bỏ các thành phần nhiễu, chuẩn hóa văn bản, chia tài liệu thành các đoạn nhỏ và xuất dữ liệu ở dạng có cấu trúc. Phần lựa chọn thông tin, tạo câu hỏi và xác nhận đáp án vẫn được thực hiện thủ công.

Trong quá trình xử lý, các thành phần không mang nội dung chính như menu, footer, thanh điều hướng, script, style và các liên kết lặp lại được loại bỏ. Sau đó, văn bản được chuẩn hóa về khoảng trắng, ký tự đặc biệt và định dạng dòng. Bước này giúp bộ tài liệu tập trung vào các thông tin thực tế thay vì chứa nhiều phần nhiễu từ giao diện website.

Tài liệu sau khi làm sạch được chia thành các chunk để phục vụ truy xuất. Nhóm sử dụng kích thước chunk là 140 từ với độ chồng lấp 30 từ. Cách chia này giúp mỗi đoạn đủ ngắn để truy xuất hiệu quả nhưng vẫn giữ được một phần ngữ cảnh giữa các đoạn liên kề.

Ngoài nội dung văn bản, nhóm lưu metadata dưới dạng JSON. Định dạng JSON giúp dữ liệu có cấu trúc rõ ràng, dễ đọc lại bằng chương trình và dễ kiểm tra thủ công. Metadata có thể bao gồm thông tin về tài liệu nguồn, chunk, số từ, mã định danh và các thông tin hỗ trợ truy vết. Nhờ vậy, khi một câu trả lời sai hoặc một chunk truy xuất chưa phù hợp, nhóm có thể quay lại kiểm tra nguồn dữ liệu tương ứng.

2.2 Tạo tập câu hỏi – câu trả lời

2.2.1 Quy trình tạo câu hỏi

Sau khi hoàn thành bước xử lý tài liệu, nhóm tạo thủ công các cặp câu hỏi – câu trả lời bằng tiếng Anh. Các câu hỏi được xây dựng trực tiếp từ nội dung trong tài liệu nguồn để đảm bảo rằng mọi đáp án đều có thể kiểm chứng được.

Khi tạo dữ liệu, nhóm đọc tài liệu đã xử lý và chọn ra các thông tin thực tế, rõ ràng, ví dụ năm thành lập, địa điểm, tên ngành, mức học phí, điểm chuẩn, chỉ tiêu tuyển sinh, tổ hợp môn, loại bằng cấp hoặc số tín chỉ. Mỗi thông tin được chuyển thành một câu hỏi tiếng Anh và một đáp án tiếng Anh tương ứng. Đáp án được viết đúng theo nội dung câu hỏi, không thêm thông tin ngoài phạm vi được hỏi.

Sau khi viết câu hỏi và đáp án, nhóm đối chiếu lại với tài liệu gốc để đảm bảo câu hỏi và đáp án khớp nhau. Với các thông tin dạng bảng như điểm chuẩn, học phí hoặc chỉ tiêu, nhóm kiểm tra kỹ đúng ngành, đúng năm học và đúng đơn vị để tránh nhầm lẫn giữa các dòng dữ liệu gần giống nhau.

2.2.2 Phạm vi câu hỏi

Bộ dữ liệu được thiết kế để bao phủ nhiều nhóm câu hỏi trong phạm vi VNU/UET. Các câu hỏi lịch sử tập trung vào năm thành lập, tiền thân của trường hoặc các sự kiện quan trọng. Các câu hỏi tuyển sinh bao gồm điểm chuẩn, chỉ tiêu, tổ hợp xét tuyển và phương thức xét tuyển. Các câu hỏi học phí tập trung vào mức học phí theo ngành, theo năm học hoặc theo tín chỉ.

Ngoài ra, bộ dữ liệu còn có các câu hỏi về chương trình đào tạo, loại bằng cấp, thời gian học, số tín chỉ và quy chế học tập. Nhóm cũng tạo một số câu hỏi dạng danh sách và câu hỏi so sánh để tăng độ đa dạng. Những câu hỏi này khó hơn câu hỏi đơn giản vì hệ thống có thể phải truy xuất nhiều thông tin hoặc đối chiếu nhiều giá trị trước khi trả lời.

Bộ dữ liệu cuối cùng gồm 360 cặp câu hỏi – câu trả lời cho tập train và 114 cặp cho tập test. Dữ liệu được lưu theo định dạng yêu cầu của bài tập, trong đó mỗi dòng câu hỏi tương ứng với một dòng đáp án tham chiếu. Ngoài định dạng văn bản, nhóm cũng lưu phiên bản JSON để giữ thêm thông tin phục vụ kiểm tra, phân tích lỗi và tái lập quá trình xử lý.

2.3 Kiểm tra chất lượng dữ liệu

Sau khi hoàn thành annotation, nhóm rà soát thủ công các cặp câu hỏi – câu trả lời bằng cách đối chiếu đáp án với tài liệu nguồn. Bước này tập trung vào việc kiểm tra câu hỏi có rõ ràng hay không, đáp án có đúng thông tin được hỏi hay không, và các chi tiết như năm học, ngành, điểm, tín chỉ hoặc học phí có khớp với nguồn hay không. Các câu hỏi về điểm chuẩn, học phí, chỉ tiêu và danh sách ngành được kiểm tra kỹ hơn vì dễ nhầm lẫn giữa các dòng dữ liệu tương tự nhau.

Để ước lượng mức độ nhất quán của annotation, nhóm sử dụng hai annotator độc lập trên 30 câu hỏi test. Tập 30 câu này được chọn theo hướng ưu tiên các trường hợp khó

hơn, ví dụ câu hỏi có nhiều đáp án, câu hỏi yêu cầu danh sách ngành, câu hỏi về học phí có đơn vị/năm học, hoặc câu hỏi có đáp án dài. Mục đích của bước này là kiểm tra xem hai người gán nhãn có thống nhất về phạm vi câu trả lời hay không.

Vì đáp án trong bài toán này là văn bản tự do, nhóm không sử dụng Cohen's kappa như bài toán phân loại nhãn rời rạc. Thay vào đó, nhóm tính ba chỉ số: exact agreement sau chuẩn hóa, soft agreement khi token-level F1 giữa hai annotator lớn hơn hoặc bằng 0.80, và token-level F1 trung bình giữa hai annotator. Script tính IAA nằm ở `scripts/compute_iaa.py`; kết quả được lưu tại `data/iaaa/iaa_results.json` và các trường hợp bất đồng được lưu tại `data/iaaa/iaa_disagreements.csv`.

Bảng 2.1: Kết quả Inter-Annotator Agreement trên 30 câu hỏi

Chỉ số	Giá trị
Số câu được kiểm tra	30
Exact agreement	16.67%
Soft agreement, token F1 ≥ 0.80	36.67%
Mean token-level F1	56.85%

Kết quả IAA tương đối thấp vì subset được chọn gồm nhiều câu hỏi khó và có nhiều cách diễn đạt đáp án hợp lệ. Ví dụ, với câu hỏi dạng danh sách, một annotator có thể liệt kê các ngành cụ thể trong khi annotator khác trả lời bằng nhóm chương trình rộng hơn. Với câu hỏi về học phí, một annotator có thể ghi đầy đủ đơn vị và năm học, trong khi annotator khác chỉ ghi giá trị tiền. Các bất đồng này cho thấy cần có guideline rõ hơn về độ dài đáp án, mức độ cụ thể và cách xử lý câu hỏi có nhiều đáp án. Sau khi so sánh hai bộ annotation, nhóm rà soát lại các trường hợp bất đồng để chuẩn hóa đáp án tham chiếu dùng cho đánh giá hệ thống RAG.

3. Phần công việc đã thực hiện

3.1 Sử dụng Embedding MiniLM + QA DistilBERT

Trong phần này, nhóm triển khai một biến thể RAG theo pipeline:

Question → MiniLM embedding → FAISS retrieve → DistilBERT QA → answer

Retriever: sentence-transformers/all-MiniLM-L6-v2

Reader: distilbert-base-cased-distilled-squad

Các file mã nguồn đã bổ sung:

- `rag/rag_minilm_distilbert.py`: triển khai đầy đủ pipeline truy xuất và đọc hiểu.
- `scripts/run_system1_pipeline.py`: chạy end-to-end và gọi script đánh giá tự động.

Đầu ra hệ thống:

- `system_outputs/system_output_1.txt`
- `system_outputs/system_output_1_trace.jsonl`

3.2 Cách hoạt động chi tiết

Bước 1 - Nạp kho tri thức: Hệ thống đọc các đoạn văn bản từ `data/processed/chunks.json`. Mỗi đoạn (chunk) được gắn metadata như `chunk_id`, tiêu đề và nguồn. Ngoài ra, hệ thống có thể bổ sung các QA fact từ `data/annotations/train_qa.json` để tăng khả năng truy xuất đúng đáp án ngắn.

Bước 2 - Biểu diễn vector bằng MiniLM: Toàn bộ chunk được mã hoá thành embedding bằng mô hình `all-MiniLM-L6-v2`. Embedding được chuẩn hoá để dùng inner product tương đương cosine similarity. Câu hỏi đầu vào cũng được mã hoá theo cùng không gian vector.

Bước 3 - Truy xuất với FAISS: Các embedding của kho tri thức được đưa vào chỉ mục FAISS (`IndexFlatIP`). Với mỗi câu hỏi, hệ thống truy xuất top-*k* chunk có độ tương đồng cao nhất. Nếu không có FAISS, mã nguồn vẫn có cơ chế fallback sang tìm kiếm bằng NumPy.

Bước 4 - Đọc hiểu bằng DistilBERT QA: Mỗi chunk truy xuất được đưa vào mô hình `distilbert-base-cased-distilled-squad` dưới dạng cặp (*question*, *context*). Mô hình dự đoán span bắt đầu/kết thúc và trích xuất câu trả lời ngắn trực tiếp từ ngữ cảnh.

Bước 5 - Chấm điểm và chọn đáp án cuối: Hệ thống tính điểm cho các candidate answer theo chất lượng span và độ liên quan của chunk truy xuất. Candidate tốt nhất được chọn làm kết quả cuối cùng cho câu hỏi.

Bước 6 - Ghi kết quả: Câu trả lời cuối cùng được ghi theo đúng định dạng yêu cầu, mỗi dòng một đáp án trong `system_output_1.txt`. Đồng thời, thông tin truy xuất và điểm số được lưu trong `system_output_1_trace.jsonl` để phân tích lỗi.

3.3 Đánh giá kết quả

Kết quả đánh giá trên tập test (114 câu hỏi) với các độ đo yêu cầu:

Bảng 3.1: Kết quả đánh giá System 1

Metric	Giá trị (%)
Exact Match (EM)	38.60
Token-level F1	55.44
Answer Recall	71.05

3.4 Sử dụng BGE Embedding + QA RoBERTa với hybrid reranking

Hệ thống sử dụng BGE làm retriever và RoBERTa làm reader, nhưng được bổ sung thêm truy xuất lai và bước chọn đáp án theo loại câu hỏi để cải thiện kết quả trên các câu hỏi có nhiều số liệu, tên ngành, mã tổ hợp hoặc danh sách đáp án.

Question $\rightarrow$ BGE embedding + TF-IDF vector $\rightarrow$ hybrid retrieve $\rightarrow$ type-aware reranking $\rightarrow$ RoBERTa QA $\rightarrow$ answer

Retriever chính: BAAI/bge-small-en

Retriever bổ trợ: TF-IDF lexical retrieval

Reader: deepset/roberta-base-squad2

File mã nguồn:

- `rag/rag_bge_roberta.py`: triển khai System 2 với BGE retriever, TF-IDF hybrid retrieval, RoBERTa reader và answer reranking.
- `scripts/run_system2_pipeline.py`: chạy end-to-end để sinh `system_output_2.txt` và gọi script đánh giá tự động.

Đầu ra hệ thống:

- `system_outputs/system_output_2.txt`
- `system_outputs/system_output_2_trace.jsonl`

Trong bước truy xuất, toàn bộ corpus được tạo từ `data/processed/chunks.json` và có thể bổ sung các QA fact từ `data/annotations/train_qa.json`. Với BGE, câu hỏi được mã hóa bằng tiền tố truy xuất `Represent this sentence for searching relevant passages:`, còn các passage được mã hóa trực tiếp từ nội dung chunk. Các embedding được chuẩn hóa để tính độ tương đồng bằng inner product.

Chỉ sử dụng dense retrieval chưa đủ ổn định với dữ liệu của bài vì nhiều câu hỏi chứa tín hiệu lexical rất quan trọng như tên ngành, mã tổ hợp xét tuyển, năm học, điểm chuẩn, học phí hoặc số tín chỉ. Vì vậy, System 2 kết hợp điểm BGE dense similarity với điểm TF-IDF cosine similarity theo công thức mặc định:

$$score = 0.70 \times score_{BGE} + 0.30 \times score_{TFIDF}$$

Cách kết hợp này giúp hệ thống giữ được khả năng bắt ngữ nghĩa của BGE, đồng thời không bỏ qua các chuỗi chính xác như D01, C00, tên chương trình học hoặc mức học phí. Sau khi truy xuất top- k chunk, hệ thống tiếp tục rerank và chọn đáp án dựa trên loại câu hỏi.

Ở bước chọn đáp án, System 2 không chỉ lấy span có điểm cao nhất từ RoBERTa. Thay vào đó, hệ thống nhận diện một số nhóm câu hỏi phổ biến như câu hỏi về ngành/chương trình, học phí, điểm chuẩn, địa điểm, tổ hợp môn, loại bằng, số lượng và câu hỏi so sánh. Với mỗi nhóm, hệ thống ưu tiên kiểu đáp án phù hợp: ví dụ câu hỏi về ngành không nên trả về số điểm, câu hỏi về học phí cần giữ đơn vị tiền và năm học, còn câu hỏi về tổ hợp môn cần trả về danh sách môn thay vì chỉ trả mã tổ hợp. Nếu một QA fact trong tập train có câu hỏi gần với câu hỏi đầu vào và cùng loại đáp án, hệ thống có thể dùng trực tiếp đáp án từ fact đó.

Thông tin truy xuất của từng câu hỏi được lưu trong file trace, bao gồm điểm hybrid, điểm BGE, điểm TF-IDF, `chunk_id`, nguồn chunk và loại câu hỏi được hệ thống nhận diện. Phần trace này giúp nhóm phân tích lỗi khi hệ thống truy xuất sai chunk hoặc chọn sai dạng đáp án.

Kết quả của System 2 trên tập test nội bộ gồm 114 câu hỏi:

Bảng 3.2: Kết quả đánh giá System 2

Metric	Giá trị (%)
Exact Match (EM)	57.02
Token-level F1	69.50
Answer Recall	85.09

3.5 Sử dụng E5 Embedding + QA RoBERTa

System 3 sử dụng mô hình truy xuất `intfloat/e5-small-v2` kết hợp với mô hình đọc hiểu `deepset/roberta-base-squad2`. Pipeline của hệ thống này là:

Question $\rightarrow$ E5 embedding $\rightarrow$ vector retrieve $\rightarrow$ RoBERTa QA $\rightarrow$ answer

Retriever: `intfloat/e5-small-v2`

Reader: `deepset/roberta-base-squad2`

File mã nguồn:

- `rag/rag_e5_roberta.py`: triển khai pipeline E5 retriever và RoBERTa reader.

Đầu ra hệ thống:

- `system_outputs/system_output_3.txt`
- `system_outputs/system_output_3_trace.jsonl`

E5 là mô hình embedding được thiết kế cho bài toán matching giữa query và passage. Khi mã hóa, hệ thống thêm tiền tố **query:** cho câu hỏi và **passage:** cho tài liệu. Các vector được chuẩn hóa để tính inner product tương đương cosine similarity. Với mỗi câu hỏi, hệ thống truy xuất top- k chunk liên quan nhất, sau đó đưa từng chunk vào RoBERTa QA để trích xuất span câu trả lời. Candidate answer cuối cùng được chọn dựa trên điểm của reader và điểm truy xuất của chunk.

Kết quả của System 3 trên tập test nội bộ gồm 114 câu hỏi:

Bảng 3.3: Kết quả đánh giá System 3

Metric	Giá trị (%)
Exact Match (EM)	50.88
Token-level F1	60.77
Answer Recall	71.93

So với System 1, System 3 đạt kết quả cao hơn ở cả EM và F1. Điều này cho thấy E5 embedding phù hợp hơn cho truy xuất dạng query-passage trong miền dữ liệu của bài. Tuy nhiên, hệ thống vẫn gặp khó khăn với các câu hỏi cần liệt kê nhiều đáp án hoặc câu hỏi yêu cầu so sánh giá trị.

3.6 TF-IDF RAG Baseline

Ngoài ba hệ thống neural RAG theo phân công, nhóm xây dựng thêm một baseline dùng TF-IDF để so sánh. Baseline này không sử dụng mô hình neural embedding mà biểu diễn câu hỏi và chunk bằng vector TF-IDF, sau đó truy xuất bằng cosine similarity.

Question $\rightarrow$ TF-IDF vector $\rightarrow$ cosine retrieve $\rightarrow$ rule-based reader $\rightarrow$ answer

Retriever: TF-IDF cosine similarity

Reader: rule-based extractive reader

File mã nguồn:

- `rag/rag_tfidf.py`: triển khai baseline truy xuất lexical và trích xuất đáp án bằng luật.

Đầu ra hệ thống:

- `system_outputs/system_output_4.txt`
- `system_outputs/system_output_4_trace.jsonl`

TF-IDF baseline đóng vai trò mốc so sánh để đánh giá xem các retriever neural có cải thiện so với truy xuất theo từ khóa hay không. Reader của baseline dùng các luật đơn giản để nhận diện loại câu hỏi, ví dụ câu hỏi về năm, điểm chuẩn, học phí, số tín chỉ, tổ hợp môn hoặc URL. Vì vậy, hệ thống chạy nhanh và không cần tải model từ HuggingFace.

Kết quả của System 4 trên tập test nội bộ:

Bảng 3.4: Kết quả đánh giá System 4

Metric	Giá trị (%)
Exact Match (EM)	49.12
Token-level F1	58.21
Answer Recall	70.18

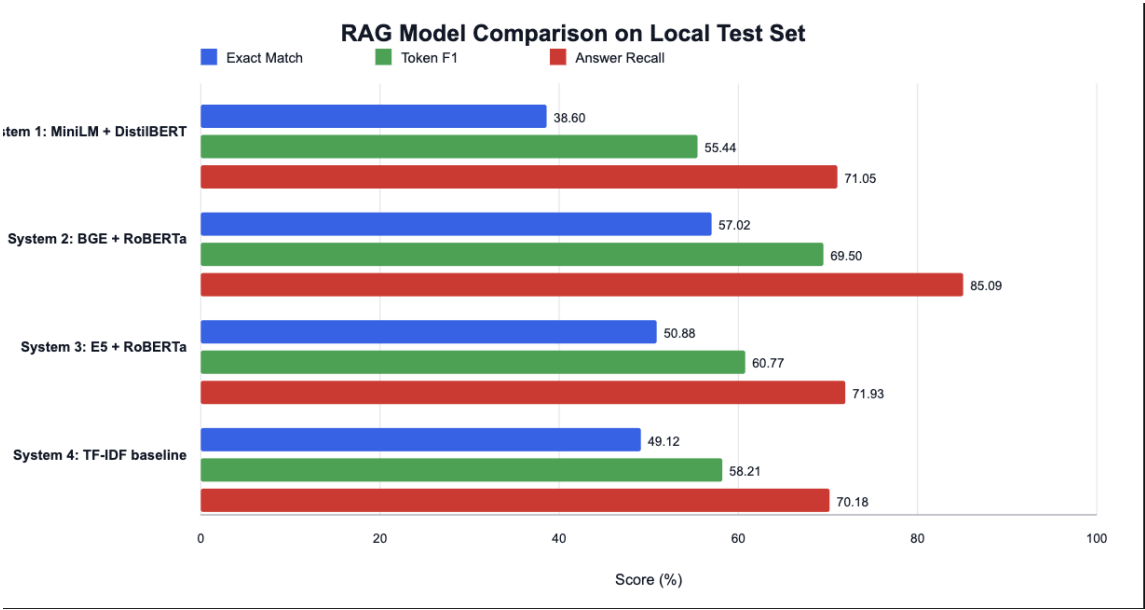
Kết quả này cho thấy TF-IDF baseline vẫn hoạt động khá tốt với các câu hỏi có overlap từ khóa cao với corpus, đặc biệt là các câu hỏi chứa tên ngành, mã tổ hợp, năm học hoặc số liệu. Tuy nhiên, baseline yếu hơn ở các câu hỏi diễn đạt khác nguồn hoặc cần hiểu ngữ nghĩa thay vì chỉ khớp từ khóa.

3.7 So sánh kết quả các hệ thống

Bảng dưới đây tổng hợp kết quả của bốn hệ thống trên cùng tập test nội bộ gồm 114 câu hỏi. Ba hệ thống đầu tương ứng với ba model chính trong phân công nhóm; System 4 là baseline để phục vụ phân tích.

Bảng 3.5: So sánh kết quả thực nghiệm

System	Mô hình	EM	F1	Recall
System 1	MiniLM + DistilBERT	38.60	55.44	71.05
System 2	BGE + RoBERTa	57.02	69.50	85.09
System 3	E5 + RoBERTa	50.88	60.77	71.93
System 4	TF-IDF baseline	49.12	58.21	70.18



Hình 3.1: Biểu đồ so sánh EM, F1 và Answer Recall giữa các hệ thống

System 2 đạt kết quả tốt nhất ở cả ba chỉ số EM, F1 và Answer Recall. Nguyên nhân chính là hệ thống này kết hợp BGE dense retrieval với lexical reranking, giúp tận dụng cả thông tin ngữ nghĩa và các tín hiệu từ khóa như tên ngành, mã tổ hợp, năm học, điểm chuẩn và học phí. System 3 dùng E5 + RoBERTa đứng thứ hai về EM, cho thấy mô hình embedding dạng query-passage có hiệu quả tốt. TF-IDF baseline thấp hơn các neural systems tốt nhất nhưng vẫn là baseline mạnh do nhiều câu hỏi trong tập test có chứa từ khóa trùng với tài liệu nguồn. System 1 có kết quả thấp nhất, chủ yếu do retrieval và reader thường chọn span chưa đúng trong các câu hỏi có đáp án dài hoặc dạng danh sách.

3.8 Nhận xét lỗi thường gặp

Qua các file trace retrieval, nhóm nhận thấy một số lỗi thường gặp:

- **Retrieval miss:** chunk chứa đáp án không nằm trong top- k , khiến reader không có đủ ngữ cảnh.
- **Wrong granularity:** hệ thống trả lời quá rộng hoặc quá hẹp so với đáp án tham chiếu.
- **Incomplete list:** câu hỏi yêu cầu nhiều ngành/chương trình nhưng hệ thống chỉ trả lời một phần.
- **Unit mismatch:** câu trả lời thiếu đơn vị như *per academic year*, *per credit* hoặc thiếu năm học.
- **Reader span error:** mô hình QA chọn span gần đúng nhưng chưa khớp chính xác với reference answer.

Các lỗi này giải thích vì sao Answer Recall thường cao hơn Exact Match: nhiều câu trả lời có chứa một phần thông tin đúng nhưng chưa khớp hoàn toàn với đáp án chuẩn.

Đóng góp của các thành viên

Thành viên	Đóng góp
Nguyễn Đức Huy	Làm knowledge base .
Nguyễn Công Cường	MiniLM + DistilBERT.
Nguyễn Trần Huy	E5 embedding + RoBERTa QA
Ngô Đình Linh	BGE embedding + RoBERTa QA